Does Rewording a Bug Report Reveal Wrong Tests? Rephrasing as a Check on AssertFlip

Zili Luo · Independent study, November 2026 · Not peer reviewed

Abstract. AssertFlip turns a bug report into a reproduction test, and its LLM validator accepts tests that a real user cannot check, because the fix does not exist yet. This study asks whether a user could find the wrong ones without the fix: run the pipeline again on meaning-preserving rewrites of the report, and distrust an accepted test that differs from the new ones. On 130 SWT-Bench-Verified bugs, with Claude Sonnet 5.5 and an analysis plan frozen before any label existed, the check did not work. Precision was 84.4% among tests that agreed and 78.1% among tests that disagreed, a gap of 6 points ($p = 0.50$), with AUROC 0.517. Rewrites were not shown stronger than plain repeat runs, and discarding disagreeing tests raised precision by 3 points at the cost of 48% of correct tests. Exploratory analysis suggests why. Outcomes cluster by bug: all five runs agree on 93 of 127 bugs, against about 36 by chance. Rewording flipped outcomes no more often than re-running did. Most disagreement was among correct tests. In a small hand read, none of six "wrong" tests was a clear misreading of the report: several followed the report where the gold patch went another way, and one never ran.

1. Introduction

A reproduction test fails on buggy code and passes once the bug is fixed. AssertFlip [1] builds one by having an LLM write a test that passes on the buggy code, flipping the assertion, and asking an LLM validator whether the failure reveals the bug. Its authors report that it beat every system on the SWT-Bench [2] leaderboard, yet 137 of its 326 accepted tests on SWT-Bench-Verified are wrong. The benchmark can tell because it holds the gold patch. A real user cannot, so an accepted test arrives with no evidence that it is right.

We would like a signal that needs no fix. MetaQA [3] detects hallucination by rewording and negating a model's answers and checking whether they hold up. CodeMetaAgent [4] rewrites a task to help generation. This study takes MetaQA's logic to AssertFlip's last stage. It does not ask whether a reworded report produces better tests. It asks whether disagreement between tests from reworded reports tells a user which accepted test to distrust.

The answer here is no. The reasons turned out to be more interesting than the failure.

2. Background and related work

AssertFlip. Two gates decide acceptance. The inverted test must fail, which is mechanical and close to automatic, since a flipped passing assertion fails. Then the validator reads the failing test and its error and judges whether the failure reveals the bug. If it refuses, a planner reviews the attempt and the pipeline starts again. The authors' no-validation run shows the validator matters: precision falls from 58.0% (189 of 326) to 47.6% (167 of 351).

Nearby work. e-Otter++ [5] uses execution feedback to generate and select reproduction tests when the correct code is missing, the same constraint as here, by a different route. SWE-Doctor [6] reports that one test often covers one facet of a bug. Beyond Fail-to-Pass [7] asks what these tests are worth to repair.

Novelty check. Three searches on 2 November 2026 found no study using issue rephrasing as a check on accepted reproduction tests, and no AssertFlip follow-up doing so. This was a short search, not a systematic review, so "not found" is the claim.

3. Research questions

- **RQ1.** Among accepted baseline tests, does disagreement across rewrites predict that a test is wrong?
- **RQ2.** Is that signal stronger than disagreement across plain repeat runs?
- **RQ3.** If a user discards disagreeing tests, how much does precision rise, and how many correct tests are lost?

Section 6 adds questions raised after labels were seen. They are exploratory and are kept apart from these three.

4. Method

Workflow. Each bug's original report went through the full, unchanged AssertFlip pipeline to give the test T0. Two rewritten reports went through it again to give T1 and T2. Two repeat runs on the original report are the control. The check compares finished tests, so it sits after AssertFlip's validator. Rephrasing touches only the report, never a step inside the pipeline.

Bugs. One order of the 433 SWT-Bench-Verified bugs, stratified by repository and fixed by seed. The study stopped at the first 130, about 30%. That stop was decided with no result seen, because the runs are costly and this is a small study, and scoring could not add bugs. All 130 baseline runs produced a test.

Rewrites. Paraphrase and step-by-step, from the report's text alone. Code, stack traces and exact values were masked before the model saw the text and restored afterwards, and a separate call judged whether the rewrite described the same bug. A rewrite had three tries. 129 of 130 bugs passed. `django-14580` failed and has no rewrite arms. Hand reads found no invented fact.

Runs. 5 arms, 648 valid runs, 647 tests, one real silent run (`sympy-17318` in repeat1). 3,564 model calls under the final prompt, 0 refusals. A run counted only if every call in it succeeded.

Changes to AssertFlip. The full diff is `assertflip-changes.diff`. Inside the code there are four changes:

Change	Where	Purpose
Deleted the instruction to reason in `<think>` tags	Test writer	Refusals, section 7
Deleted the `<think>` block from the output format	Test writer	Same
"THOUGHT PROCESS" relabelled "PREVIOUS PLAN"	Retry planner	Accuracy: the value was always the previous plan
Save the attempt log when a run fails	`assertFlip.py`	Logging only

Outside the code:

	Authors	This study
Model	GPT-4o	Sonnet 5.5, thinking on, `medium` effort, default temperature
Regenerations	10	5, the authors' own ablation value; one run used all 5
Run timeout	1 hour	2 hours; longest run 945 s
Prompts	Real turns	Flattened to one message, a fresh `claude -p` per call
`get_info`	Real tool	Text convention, run by AssertFlip in the container
Temperature, `max_tokens`	Enforced	Not enforced
Runner	`run_parallel.py`	Same steps; only the four fields AssertFlip reads reach the container
Rewrite arms	n/a	Localization stays as computed from the original report

Model calls used a host-side service speaking the OpenAI format, so `openai/gpt-4o` remains in the configuration as a label only.

Label. A test is right if SWT-Bench reports it `resolved`, otherwise wrong, using the unmodified harness at commit `eee3862` in unit test mode with the authors' path map. This is stricter than plain fail-to-pass: the test must go fail-to-pass and add no new failing test. It is the label behind AssertFlip's published figures.

Score. A test becomes the set of things it builds, calls and expects, with local names, comments and import style removed. Distance between two tests is Jaccard distance, and a test's score is its mean distance to the two other tests. The scorer was fixed before any label existed, after two noise rules were tuned on repeat pairs only. The runtime half (one line of failure text) is reported as secondary: on repeat pairs it added more assertion style than signal, and it had never been tested. A free pretest on the authors' GPT-4o variant runs passed its gate, with AUROC 0.688, CI [0.614, 0.759]. Plain text distance is the comparator throughout, because it did nearly as well there (0.679).

Plan. `analysis-plan.md` was frozen before any label existed. Agree is a score at or below the median, disagree above it. RQ1's test is Fisher's exact on precision, with repository-stratified AUROC secondary, because repository strongly predicts outcome. RQ2 compares AUROC with a paired bootstrap. RQ3 is descriptive. A null is reported as "no large effect", since 130 bugs detect a 30-point gap about 92% of the time or better, and a 20-point gap only about 58-91%. Fewer than 20 wrong or 20 right tests would have been inconclusive.

Safety. Downloaded code ran only in containers. Tests were read as text on the host.

5. Results

642 of 647 tests have a result. The five without are `astropy-8872` in every arm: its image can no longer be built, because an unpinned `setuptools` now lacks `pkg_resources`. Three other first-pass failures cleared on the plan's one retry. So RQ1-3 run on 128 bugs (130 less `django-14580` and `astropy-8872`), with 104 right, 24 wrong, precision 81.2%. That clears the floor of 20. This figure is not comparable with any published number and no such claim is made.

RQ1: no large effect.

Group	Tests	Right	Wrong	Precision
Agree	64	54	10	84.4%
Disagree	64	50	14	78.1%

The gap is 6.2 points (Fisher p = 0.50). Stratified AUROC is 0.517, CI [0.306, 0.725], about chance. A 30-point gap is ruled out. The interval is wide, so a modest signal is not.

RQ2: not shown stronger. Rewrite AUROC 0.517, repeat AUROC 0.506, difference +0.011, CI [-0.135, 0.166]. Repeats had the larger gap, 9.4 points (p = 0.26). Plain text distance gave 0.468, and the runtime half 0.533 with the same split.

RQ3: a poor trade.

Discard	Precision before	Precision after	Correct tests lost
Rewrite disagreement	81.2%	84.4%	50 of 104 (48%)
Repeat disagreement	81.2%	85.9%	49 of 104 (47%)
Text distance	81.2%	82.8%	51 of 104 (49%)

Discarding a random half would remove about 12 wrong tests and lose about 52 right ones. The filter removed 14 and lost 50, so it beat a coin flip by about 2 tests. For contrast, the same cut in the GPT-4o pretest, with the current scorer, raised precision by 15 points and lost 37% of correct tests.

Sensitivity lines do not change a verdict. With `added_f2p > 0` as the label the gap is 6.2 points, and with `astropy-8872` counted wrong it is 5.0.

Hand read. The plan's seeded sample of 12, read with the report, tests, failure, harness output and gold patch. The six right tests reproduce their reports, and the three in the disagree group differ only in technique. None of the six wrong tests is a clear misreading:

Bug	Why it is labelled wrong	Runs wrong, of 5
`sphinx-8551`	Never ran at scoring. `roman` is missing from the scoring image, before and after the fix†	5
`sympy-21930`	Partly right. One function goes fail-to-pass; another holds a control that the gold patch changes	5
`astropy-14309`	Partly right. Five cases pass after the fix; an extra `filepath=None` case does not	2
`pydata-4687`	The report says attributes "should be preserved or at least there should be a choice"; gold adds a choice and keeps the default	5
`django-15695`	The report says backward "should restore the old auto-generated name"; gold makes forward a no-op instead	5
`django-12273`	Likely the same: the report sets the field directly, while gold changes the primary-key setter. Read, not confirmed by running	5

† An environment difference between containers, not a fault in the test. AssertFlip ran in a prebuilt container with docutils 0.21.2. The scoring container was built fresh, the benchmark's environment does not pin docutils, and it received 0.23, which no longer ships `docutils.utils.roman`. The same applies to `sphinx-8721`. The true label of these tests is unknown; see section 7.

Across all 24 wrong baseline tests, 2 are the `roman` crash (`sphinx-8551`, `sphinx-8721`), 6 are partly right by the harness's own count, and 16 are neither, of which three were read. This is not a rate. Labels were not changed. Dropping the two crashed bugs gives a gap of 3.2 points ($p = 0.82$) and leaves AUROC at 0.517.

Gold check, after the fact. Each harness result also records the benchmark's own gold test. On 9 of 129 bugs it shows no fail-to-pass in the scoring images: 5 matplotlib, 2 pylint, the 2 sphinx bugs above. On 4 of the 9 our test still resolved, so the labels there are not in doubt from this. The other 5 are labelled wrong and are suspect: the two crashes, plus `matplotlib-26291`, `pylint-6386` and `pylint-7080`, where the cause was not traced. Labels stand as planned. Counting all five right gives a gap of 4.7 points ($p = 0.62$). The case most favourable to the signal, the two close bugs right and the three far bugs wrong, gives 9.4 points ($p = 0.24$). No outcome for these five changes a verdict. Dropping all five leaves 19 wrong tests, under the plan's floor of 20, which is a reason to read that line as a sensitivity only.

6. Exploratory analysis

Everything here was raised after labels were seen. It is not in the frozen plan and no result here is confirmatory.

When T0 is far from T1 and T2, are they right? Usually yes.

T0	Bugs	T1, T2 both right	One right	Neither right
Far, T0 right	50	37	11	2
Far, T0 wrong	14	2	5	7
Close, T0 right	54	48	5	1
Close, T0 wrong	10	1	0	9

In 37 of the 64 far cases all three tests are right, so disagreement mostly marks several correct tests. In 9 of the 10 close-but-wrong cases all three are wrong. Repeats show the same shape. "Right" means each passes the harness, not that they check the same behaviour.

Does the outcome depend on the bug? On the 127 bugs with all five results, 80 are right on all five runs, 13 are wrong on all five, and 34 are mixed. The 93 unanimous bugs compare with 36.5 when labels are shuffled across bugs (95% range 30-43) and 46.0 when shuffled within a repository (40-52). None of 10,000 shuffles reached 93. The intraclass correlation is 0.617. This shows clustering, not its cause. The hand read suggests part of the cause sits in the gap between report and gold patch.

Does another run get it right? With T0 wrong, T1 or T2 is right on 7 of 14 far bugs and 1 of 10 close bugs ($p = 0.079$, post hoc). Using all four other runs it is 8 of 14 and 3 of 10 ($p = 0.24$), and 11 of 24 overall. This is a hint at most, and a user still could not tell which run to keep.

Is there too little room to reword? No evidence for that. The median report has 141 words, of which 87 were free to change, and rewrites changed 43% (paraphrase) and 66% (step-by-step) of that prose at word level. Free words do not relate to the number of right runs (Spearman -0.03), and the amount reworded does not relate to flips (-0.12, $p = 0.17$).

Does wording move outcomes? Against T0, outcomes flipped on 13.0% of comparisons under rewrites and 13.8% under repeats, a difference of -0.8 points, CI [-5.1, 3.5]. Right rates per arm are

81.4% baseline, 78.1% paraphrase, 75.8% step-by-step, 80.5% and 75.2% for repeats, so the rewrites sit inside the 5-point spread of plain repeats. Results were robust to wording. That is not evidence that the model understands the report: 13 bugs are stably wrong, and understanding was not measured.

How often did the validator refuse? Rarely: 22 of 669 calls, 3 of 133 on baseline. 17 of 648 runs reached regeneration, 2 of them baseline. All 24 wrong baseline tests passed it. The cause is unknown and the port may contribute. The public package has no per-call data, so this rate cannot be compared with GPT-4o's.

7. Practical findings for anyone porting AssertFlip

Refusals. Under the original prompt, Sonnet 5.5 returned `stop_reason: "refusal"` on ordinary code: 2 refusals in about 10 test-writing calls. The one category captured was `[reasoning_extraction]`, which Anthropic defines as a request to reproduce internal reasoning in the reply, listing a fixed `<thinking>` section as a typical cause. GPT-4o has no separate reasoning stage, so the same line means something different on each model. The `<think>` instruction and block were **deleted, not reworded**, and nothing replaced them. No alternative wording was tested, because sending variants to see what passes is what the safeguard rule forbids, so this report cannot say which wording is safe. Afterwards there were 0 refusals in 3,564 calls, though only 17 runs reached the retry planner. This is observational. Replies are kept in message history, so GPT-4o's later turns could see its written reasoning, which is a port difference; none of the logged Sonnet replies held the tag. When porting a prompt, remove any request to write reasoning in the reply, and treat a refusal as a prompt problem to fix.

Scoring drift. Images built today differ from those built when the authors scored. One bug no longer builds, and two sphinx bugs crash for missing `roman`, even with the gold test. Our runs used prebuilt images, where those tests ran normally: on `sphinx-8721` the prebuilt image has docutils 0.21.2, which ships `docutils.utils.roman`, and the image built at scoring has 0.23, which does not. The model never saw this error, because scoring gives the pipeline no feedback, and the true label of these tests is unknown. The harness's own cleanup looks for `sweb.*` names while its images are `exec.*`, so it removes nothing, and about 190 GB of images remained. Check the gold test on every bug before trusting scores.

8. Discussion

The check failed here for two reasons that fit together. Correct tests differ from each other, because one bug can be tested well in many ways, so distance mostly measures that variety. Wrong tests tend to agree, because on most bugs every run lands on the same outcome. So the filter discards correct tests that merely differ and keeps wrong tests that agree. Since rewording moved outcomes no more than re-running did, a wording-based check had little to reveal on this model.

The pretest signal did not carry over. It came from GPT-4o and from runs that differed in configuration, not wording, and its wrong tests were far more common. Which of those differences matters is unknown, and this study cannot separate them. Passing a pretest justified the spend. It did not predict the result, and the pretest's own limits said so.

The hand read raises a harder point. "Wrong" here means "disagrees with the gold patch", and that is not always "misunderstood the bug". A test can follow the report faithfully and still fail after a fix that

took another route. If that is common, part of the wrong-test rate measures the gap between reports and fixes, and no check that lacks the fix can close it: neither the validator, nor disagreement. Six tests cannot say how common it is.

For a user, the practical answer is simple. On this evidence, do not run AssertFlip three times to filter its output.

9. Threats to validity

- **Few wrong tests.** 24 wrong tests limit every analysis, and precision near 81% leaves the agree group little room to rise.
- **Label.** Two wrong labels are crashes, three more are suspect, others may reflect report and gold differences, and partly right tests count as wrong under the primary label. The labels stand as planned, and the sensitivity lines change no verdict.
- **Scoring check.** The gold test was run on one bug before scoring. Running it on every bug would have shown the 9 bugs in section 5 before any label existed, and anyone reproducing this should do that first.
- **Port.** Flattened prompts, `get_info` by convention, and prompts tuned for GPT-4o. `claude -p` adds context noise, including the account email and paths, the same in every arm. Each call is a fresh process. Effort stayed at `medium`, and thinking is on, unlike GPT-4o.
- **Scorer.** Static, and it treats a helper call as a difference. Most repeat disagreement survived two noise fixes.
- **Rewrites.** Wording changed by design and facts did not, so nothing here speaks to changing a report's information. Word-level change counts reordering as change.
- **One run.** Each arm ran once per bug, and the harness ran each test once.
- **Exploratory work.** Several analyses followed the data, so their p-values and intervals are descriptive.
- **Sample.** One tool, one model, 130 of 433 bugs. This says nothing about rephrasing in general.

10. Reflection

The pretest passed, and the main study returned something close to a coin flip. Freezing the plan first meant there was nothing to adjust, and I had already decided, before any label existed, that a negative result would be reported as plainly as a positive one.

The most useful questions came after the verdict. Asking why the procedure looked strange led to checking whether T1 and T2 were right, which showed that most disagreement sat among correct tests. That was not in the plan, and it explains more than the plan's own tests did.

I also carried a wrong picture for part of the study, thinking of the work as asking whether rephrasing improves AssertFlip's tests, and of its validator as a check on the inversion. Both had to be corrected against the code and the spec. Reading the harness output by hand changed the reading again: a number called "wrong" had several different things under it, including a test that never ran.

Finally, a port is a study in itself. Most of what surprised me here, the refusals and the drifted images, came from running someone else's system on a new model and on a benchmark whose dependencies had moved, not from the question I set out to ask.

11. Future work

Several directions look worth exploring, and none is a fixed plan.

- **Rephrasing inside AssertFlip's steps.** This study rephrased only the report, outside the pipeline, and compared finished tests. Using rewrites inside a step, such as the planner or the validator, was not tested, and this negative result says nothing about it.
- **Report and gold differences.** How many wrong labels are tests that follow the report where the fix went elsewhere?
- **Changing the information.** Rewrites here kept every fact. Removing or varying the code in a report would show what the model relies on.
- **Measuring understanding.** Ask for a stated root cause and compare it with the gold patch.
- **Retry on disagreement.** Does distance mark the bugs where another run can succeed? The hint here is weak.
- **Choosing among runs.** Some bugs have right and wrong runs, and nothing yet tells a user which is which.
- **The validator on newer models.** Why does it refuse so rarely here, and how much of that is the port?
- **Other models.** The same check on GPT-4o rewrites would separate the model from the kind of variation.

12. Conclusion

On 130 bugs, rewording a report and comparing the resulting tests did not tell correct accepted tests from wrong ones, did not beat plain re-running, and was a poor filter. Outcomes depended far more on the bug than on the wording, which leaves a wording-based check little to see. The most useful result may be the caution it leaves behind: a wrong label on a reproduction test can mean the test disagrees with one particular fix, and any check that lacks that fix will struggle to see the difference.

How this was built

After reading the AssertFlip paper I found its hypothesis convincing: LLMs tend to make things pass, and are better at that than at making them fail on purpose. With the insight from MetaQA, I hypothesized that rephrasing an issue might change the actual output, and that comparing the outputs could show which tests to distrust. After the initial research questions, I raised RQ4 and the bug effect, then asked whether short reports or the model's understanding could explain the result. I also decided to report the refusal as a finding, to stop at 130 bugs, and to report a negative result as plainly as a positive one.

Claude Code assisted me in sharpening the research points and in building and running the test environment. It also wrote the code and carried out the hand read, and it drafted this report under my direction. I reviewed the results, decided what the report should say and directed the revisions. AssertFlip is the work of Khatib, Mathews and Nagappan, and this study builds on their published code and data.

Reproduce

`scripts/` holds the pipeline, scorer, analysis and harness wrapper, with paths pointing at a local working folder. `results/` holds the plan, the pretest plan and all outputs. `assertflip-changes.diff` lists every change to AssertFlip. Generated tests and raw run logs are not in this repo.

References

1. L. Khatib, N. S. Mathews, M. Nagappan. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests. arXiv:2507.17542, v2, 2026. Code and data: github.com/uw-swag/AssertFlip.
2. N. Mündler, M. N. Müller, J. He, M. Vechev. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. NeurIPS 2024. arXiv:2406.12952.
3. Yang, Al Mamun, Zhang, Uddin. Hallucination Detection in Large Language Models with Metamorphic Relations. FSE 2025. arXiv:2502.15844.
4. Akhond, Uddin. LLM Assisted Coding with Metamorphic Specification Mutation Agent. arXiv:2511.18249, 2025.
5. T. Ahmed, J. Ganhotra, A. Shinnar, M. Hirzel. Heterogeneous Prompting and Execution Feedback for SWE Issue Test Generation and Selection. ICSE 2026. arXiv:2508.06365.
6. Y. Guo, Y. Liu, J. M. Zhang, Y. Ma, Y. Lou, Z. Chen. SWE-Doctor: Guiding Software Engineering Agents with Runtime Diagnosis from Multi-Faceted Bug Reproduction Tests. arXiv:2607.00990, 2026.
7. Y. Tan, Z. Yang, F. Yang, Y. Yao, Y. Kang, L. Wang, P. Zhao, X. Zhang, X. Ma, Q. Lin, S. Rajmohan, D. Zhang. Beyond Fail-to-Pass: Iterative Hardening of Co-Generated Bug Reproduction Tests and Fixes. arXiv:2607.19843, 2026. Read only in part.